ESTUDIO
PELÍCULAS

Proyecto Data Analytics II

Pelayo Ocaña Camacho

Índice

1. Contexto e introducción.
2. Metodología.
3. Implementación técnica.
   3.1 División Datos.
   3.2 Filtrado Basado en contenido.
   3.3 Filtrado colaborativo.
4. Evaluación y resultados.
5. Análisis y conclusiones.
6. Limitaciones.

1. Contexto e introducción.

Lumière Studios, con más de dos décadas de experiencia en la industria cinematográfica, enfrenta hoy el desafío de adaptarse a una nueva era de consumo audiovisual. Las plataformas digitales y el acceso inmediato a contenido han transformado los hábitos de nuestros clientes. Conocer en profundidad sus preferencias es clave para ofrecer una experiencia personalizada y mantenernos como líderes en el mercado.

En este contexto, la implementación de sistemas de recomendación se vuelve una herramienta estratégica para anticipar gustos, optimizar la promoción de películas y fidelizar audiencias.

Dentro de los objetivos perseguidos en este análisis podemos destacar la construcción de sistema que predigan con alta precisión las preferencias cinematográficas de nuestros clientes, mejorar la oferta personalizada de títulos en catálogos y promociones y aumentar la retención y satisfacción de usuarios a largo plazo.

2. Metodología

Utilizamos el conjunto de datos MovieLens ml-latest.zip, ampliamente reconocido en la investigación de sistemas de recomendación.

El proceso se dividió en varias etapas: división de datos en conjuntos de entrenamiento (80%) y prueba (20%), implementación de dos enfoques: filtrado basado en contenido (FBC) y filtrado colaborativo basado en usuarios (FCU), evaluación de la precisión mediante la métrica RMSE, análisis comparativo de resultados y discusión de implicaciones.

3. Implementación técnica.

División de Datos

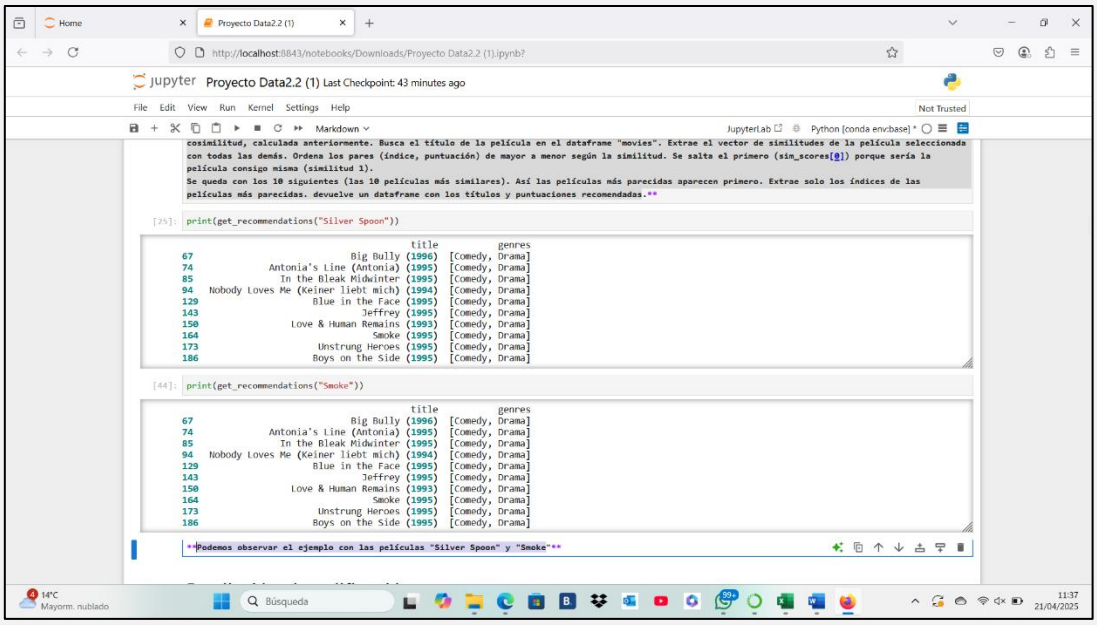
Comenzaríamos importando las librerías que vamos a necesitar, en este caso usaremos: pandas, numpy, MultiLabelBinarizer, cosine_similarity, train_test_split, mean_squared_error, matplotlib, math y rich. Dichas librerías nos servirán para llevar a cabo la manipulación de los datos y la impresión de estos de manera más estética.

Hemos cargado el archivo csv "ratings", necesario para poder el filtrado. Este conjunto lo dividiremos en dos subconjuntos internos; uno de entrenamiento(train), que será el 80% del total y el 20% restante será el correspondiente al subconjunto de prueba(test). Asegurando un registro para cada uno de os individuos registrados en el csv. Al tener un registro de prueba y entrenamiento por cada individuo nos asegura una coherencia para corroborar las predicciones. A continuación, cargo el segundo csv necesario que será "movies", en el cual cada fila es una película. Extraemos los géneros de cada película, y los agrupamos en listas. Empleamos MultiLabelBinarizer para convertir las listas de etiquetas múltiples en matriz de 1 y 0, llamado también "One-Hot enconding". Tomará valor 1 cuando la pelócula tenga ese género y 0 cuando no. Crea un índice con los números identificadores de cada película, de tal forma que nos permita acceder más fácilmente a estas. Después, calculamos la similitud entre las películas, usando la matriz de consimilitud, tomando 1 cuando haya géneros en común y 0 cuando no.

Filtrado Basado en contenido

Comenzamos definiendo una función "**get_recommendations",** donde tomará como parámetros el título de la película y la matriz de cosimilitud, calculada anteriormente. Busca el título de la película en el dataframe "movies". Extrae el vector de similitudes de la película seleccionada con todas las demás. Ordena los pares (índice, puntuación) de mayor a menor según la similitud. Se salta el primero (sim_scores[0]) porque sería la película consigo misma (similitud 1).
Se queda con los 10 siguientes (las 10 películas más similares). Así las películas más parecidas aparecen primero. Extrae solo los índices de las películas más parecidas. devuelve un dataframe con los títulos y puntuaciones recomendadas.

Podemos observar el ejemplo con las películas "Silver Spoon" y "Smoke”, siendo “Big Bully” y “Jeffrey”, recomendadas en ambos casos.



La función **predict_content** predice qué calificación (rating) le daría un usuario a una película basándose en las películas que ya ha visto y en la similitud de géneros entre películas.

Primero, verifica si la película existe en el sistema; si no, devuelve NaN. Luego, obtiene todas las películas que el usuario ya calificó en el conjunto de entrenamiento, y calcula cuánto se parece cada una de esas películas a la nueva película usando la matriz de similitud coseno (cosine_sim).

Solo guarda las similitudes y calificaciones de las películas que también existen en el sistema. Si no hay ninguna película comparable, devuelve NaN. Después convierte las similitudes y calificaciones en arrays de numpy y selecciona las k películas más parecidas a la nueva, ordenándolas por similitud.

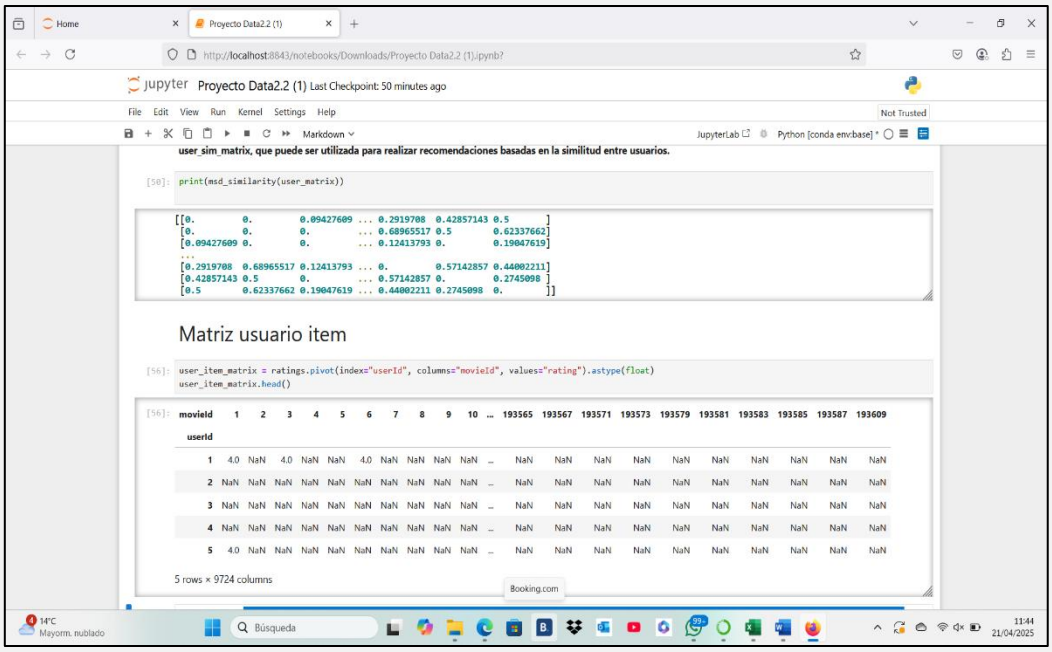
Finalmente, predice el rating calculando un promedio ponderado de las calificaciones, donde las películas más similares tienen más peso; si la suma de las similitudes es cero (es decir, si no hay nada muy parecido), simplemente hace un promedio normal de las calificaciones de esas k películas. Así, se logra estimar de manera inteligente qué tan probable es que al usuario le guste la nueva película basándose en su historial de gustos y en el parecido de los contenidos

Filtrado Colaborativo

Comenzamos con la elaboración de la **matriz MSD**, una matriz de interacción entre usuarios y películas a partir del conjunto de entrenamiento (train_ratings), utilizando el método pivot de pandas para construir una matriz en la que las filas representan a los usuarios, las columnas a las películas, y los valores a las calificaciones que los usuarios han dado a las películas, convirtiendo los valores a tipo float.

Luego, genera un diccionario que mapea cada userId a un índice único dentro de la matriz. Después, la función msd_similarity(matrix) calcula la similitud entre usuarios utilizando el Mean Squared Difference (MSD). Para cada par de usuarios, compara las calificaciones que ambos han dado a las mismas películas (comunes), calcula la diferencia al cuadrado entre sus calificaciones, y luego obtiene la media de esas diferencias al cuadrado. La similitud entre los usuarios se calcula como el inverso de 1 más el MSD, de forma que valores más bajos de MSD resultan en una mayor similitud. Si no hay películas comunes entre dos usuarios, la similitud se establece en 0. Finalmente, la función devuelve una matriz de similitudes entre todos los usuarios, user_sim_matrix, que puede ser utilizada para realizar recomendaciones basadas en la similitud entre usuarios.

Matriz **usuario-item,** una matriz de usuario-película utilizando el método pivot de pandas, transformando el DataFrame de calificaciones (ratings) en una tabla donde las filas corresponden a los usuarios (userId), las columnas a las películas (movieId), y los valores de la tabla son las calificaciones (rating) que los usuarios han dado a las películas. Luego, mediante. astype(float), asegura que todos los valores de la matriz sean de tipo float, lo cual es necesario para realizar cálculos matemáticos y de similitud en el análisis de datos. Finalmente, user_item_matrix.head() muestra las primeras 5 filas de la matriz resultante, permitiendo inspeccionar cómo se distribuyen las calificaciones entre usuarios y películas. Este tipo de matriz es fundamental en sistemas de recomendación, ya que facilita la comparación de patrones de calificación entre usuarios y películas, y es utilizada en técnicas como el filtrado colaborativo, donde la similitud entre usuarios o entre películas es calculada para generar recomendaciones personalizadas.



Definimos la función **msd_similarity,** la cual toma como entrada una matriz que representa las calificaciones de los usuarios para diferentes películas, donde las filas corresponden a los usuarios y las columnas a las películas. La variable users obtiene los índices (identificadores) de los usuarios y n_users almacena el número total de usuarios. Se inicializa una matriz de similitud (sim_matrix) de tamaño n_users x n_users llena de ceros. Luego, la función recorre todos los pares de usuarios (incluyendo cada usuario consigo mismo). Si los usuarios son el mismo (i == j), asigna una similitud perfecta de 1.0 en la diagonal de la matriz. Si los usuarios son diferentes (i < j), calcula la similitud de la siguiente manera: primero, identifica las películas comunes que ambos usuarios han calificado (es decir, aquellas que no son NaN para ambos usuarios). Si no tienen películas comunes, asigna una similitud de 0.0. Si tienen películas comunes, calcula la diferencia cuadrada entre sus calificaciones para esas películas, promedia las diferencias cuadradas y aplica la fórmula de similitud sim = 1 / (1 + MSD), donde el MSD es la media de las diferencias cuadradas. Después, asigna este valor de similitud tanto en la posición [i, j] como en [j, i] de la matriz de similitud, ya que la similitud es simétrica. Finalmente, la función devuelve la matriz de similitud como un DataFrame de pandas con los identificadores de usuario como índices y columnas.

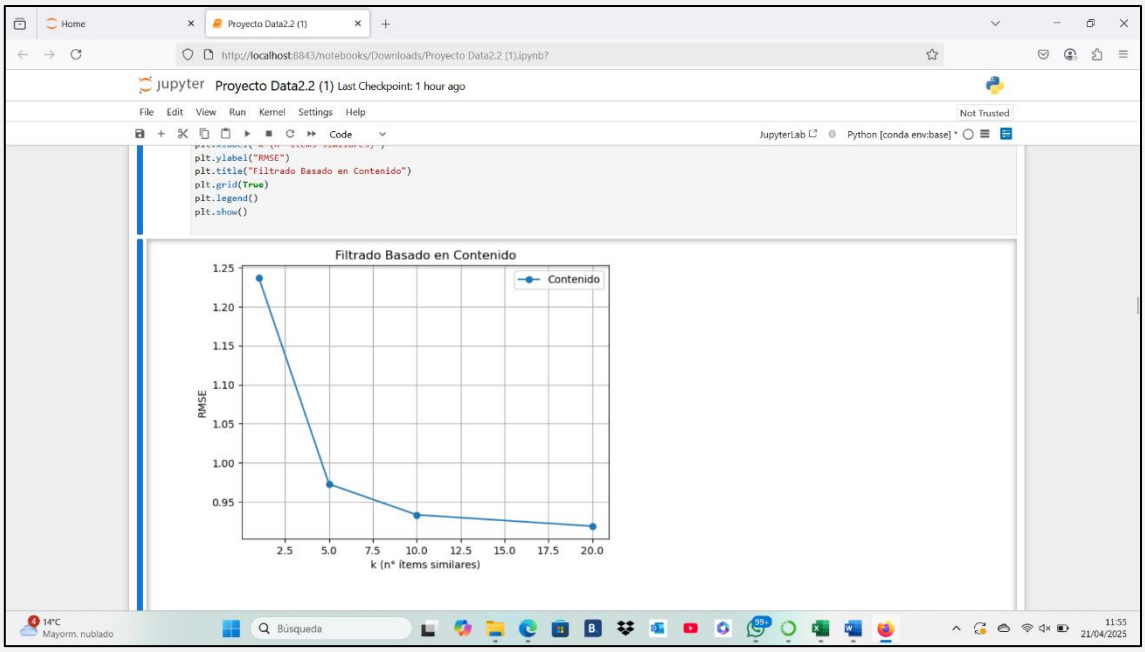
El valor 0.500000 entre el usuario 1 y el usuario 2 indica que hay una similitud moderada entre los dos en sus calificaciones.
El valor 0.094276 entre el usuario 1 y el usuario 3 indica que los patrones de calificación entre estos dos usuarios son bastante diferentes.

Definimos la función **predict_collaborative**, realiza una predicción de la calificación que un usuario daría a una película utilizando un enfoque de filtrado colaborativo basado en la similitud entre usuarios. Primero, verifica que el user_id y movie_id sean válidos, es decir, que el usuario exista en la matriz de usuarios y que la película haya sido calificada por al menos un usuario. Luego, obtiene las similitudes entre el usuario objetivo y todos los demás usuarios, utilizando una matriz de similitudes realizada anteriormente (user_sim_matrix). A continuación, identifica los usuarios que han calificado la película y extrae tanto las similitudes de esos usuarios con el usuario objetivo como las calificaciones correspondientes para esa película. Si existen usuarios con calificaciones, la función selecciona los k usuarios más similares (basado en sus similitudes), y calcula la predicción ponderando sus calificaciones por la similitud de cada usuario, usando la fórmula de la media ponderada. Si la suma de las similitudes es cero (lo que indicaría que no hay usuarios similares), devuelve la media de las calificaciones de esos usuarios para la película. Este enfoque de predicción permite recomendar películas basadas en el gusto de usuarios similares al usuario objetivo, utilizando un método clásico de filtrado colaborativo basado en similitudes entre usuarios.

4. Evaluación y resultados.

RMSE filtrado basado en contenido.

Evalúa la calidad de las predicciones hechas por el modelo de filtrado basado en contenido utilizando la métrica RMSE (Root Mean Squared Error).
Para cada interacción en el conjunto de prueba (test_ratings), la función predice la calificación que un usuario daría a una película usando predict_content(userId, movieId, k), donde k es el número de películas similares a considerar; si la predicción es válida (no NaN), guarda tanto la predicción como la calificación real.



Luego, calcula el RMSE entre las calificaciones reales y las predichas, que mide qué tan cerca están en promedio las predicciones del modelo respecto a los valores reales (un RMSE más bajo es mejor). Después, se define una lista de valores de k ([1, 5, 10, 20]) y se calcula el RMSE para cada uno usando la función evaluate_rmse_content.

Finalmente, se genera una gráfica donde el eje X representa los diferentes valores de k (el número de películas similares que se usan en la predicción), el eje Y muestra el RMSE correspondiente, y se dibuja la curva para visualizar cómo cambia el error a medida que se usan más vecinos en las predicciones.

Así, puedes analizar visualmente qué valor de k ofrece el mejor equilibrio entre precisión y generalización para tu sistema de recomendación basado en contenido.

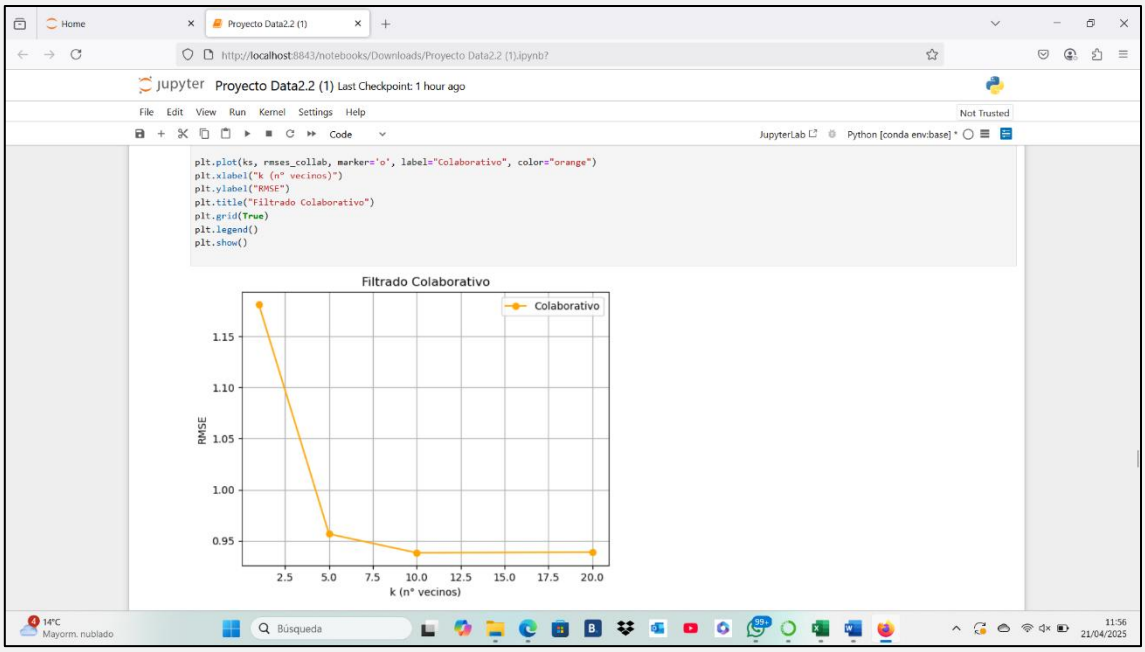
Podemos observar como a medida que aumentan los k, mejora el rmse. Sin embargo, en caso de tener un k excesivamente alto, podría dar lugar a subajuste y una falta de exactitud en la predicción. Siendo opuesto cuando el k es demasiado pequeño, por ejemplo 1. En este caso, el modelo se sobreajuste a muy pocas películas similares y dar lugar a una alta variabilidad.

RMSE filtrado colaborativo.

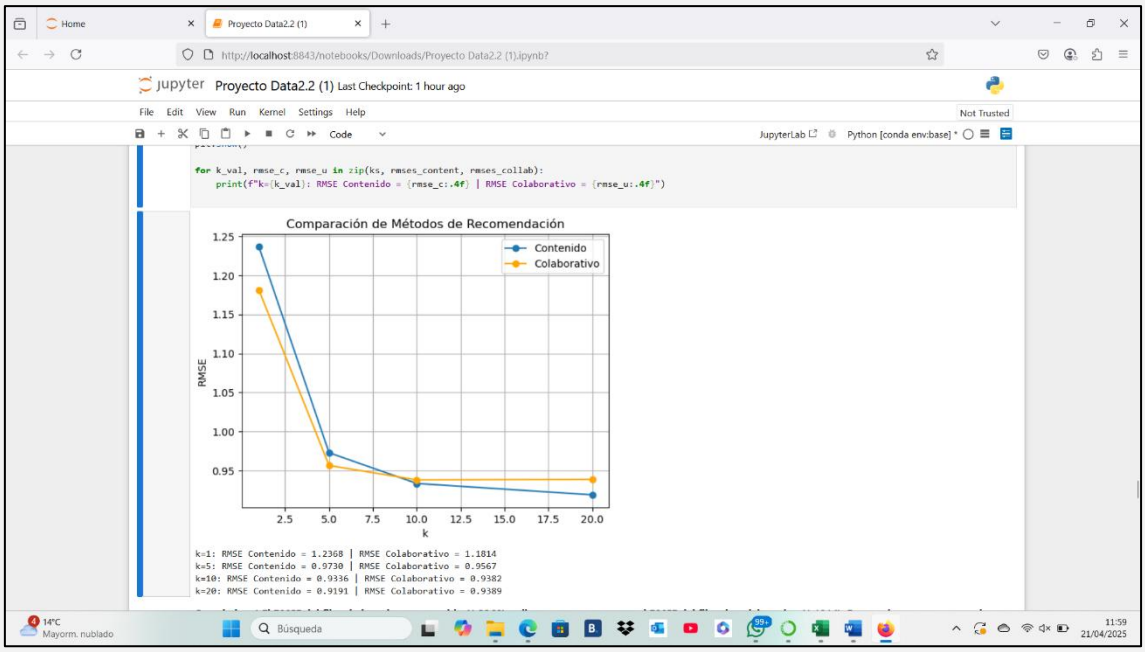
Evalúa el desempeño de un modelo de filtrado colaborativo midiendo su precisión con el RMSE (Root Mean Squared Error), utilizando el número de vecinos más cercanos k para hacer las predicciones. Para cada valor de k, recorre el conjunto de prueba test_ratings, predice las calificaciones que un usuario daría a una película con la función predict_collaborative, y calcula el error entre las calificaciones predichas y las reales.

Si la predicción es válida (no NaN), se agrega a las listas de predicciones y calificaciones reales. Después, la función calcula el RMSE usando la raíz cuadrada del error cuadrático medio. Se evalúa este RMSE para diferentes valores de k (como 1, 5, 10, 20), y los resultados se grafican, mostrando cómo cambia el RMSE según el número de vecinos considerados.

El gráfico generado ayuda a visualizar el impacto de k en la precisión del modelo de filtrado colaborativo, permitiendo identificar el valor óptimo de k para minimizar el error y mejorar las predicciones.



5. Análisis y conclusiones.



- Cuando k = 1.El RMSE del filtrado basado en contenido (1.2368) es ligeramente mayor que el RMSE del filtrado colaborativo (1.1814). Esto sugiere que, con un solo vecino más cercano, el modelo de filtrado colaborativo está haciendo predicciones más precisas en comparación con el modelo basado en contenido.

- Cuando k = 5. Ambos métodos muestran una mejora en sus resultados, ya que los RMSE disminuyen en comparación con el valor de k = 1. El RMSE del filtrado colaborativo (0.9567) sigue siendo más bajo que el del filtrado basado en contenido (0.9730), lo que indica que el filtrado colaborativo sigue teniendo un rendimiento ligeramente mejor con un número mayor de vecinos.

- Cuando k = 10. A este valor de k, el RMSE del filtrado basado en contenido (0.9336) es un poco más bajo que el RMSE del filtrado colaborativo (0.9382), lo que sugiere que, en este caso, el modelo basado en contenido ha alcanzado una mayor precisión en las predicciones que el colaborativo.

- Cuando k = 20. Los RMSE de ambos modelos se aproximan, pero el filtrado basado en contenido (0.9191) sigue siendo ligeramente mejor que el filtrado colaborativo (0.9389). Esto podría indicar que, con más vecinos, el modelo basado en contenido sigue siendo más eficiente en este conjunto de datos, o que el filtrado colaborativo no ha mejorado significativamente al aumentar el número de vecinos

- Filtrado colaborativo muestra un rendimiento mejor cuando k = 1 y sigue siendo competitivo hasta k = 5. Sin embargo, al aumentar k, el filtrado basado en contenido parece tener un mejor desempeño, especialmente a partir de k = 10, lo que sugiere que el modelo basado en contenido podría ser más robusto a medida que se consideran más vecinos. El rendimiento de ambos modelos depende del valor de k, y no siempre uno es claramente superior al otro en todas las configuraciones.

6. Limitaciones.

Dentro del estudio hemos encontrado una serie de limitaciones o dificultades, como puede ser la falta de matices capturadas por los

géneros registrados o variaciones en los registros y cantidad de estos que hacen más sensible el estudio y los métodos aplicados.